

오늘의 작업 기록 — 2026년 6월 19일

개발 환경 구축 첫 날. *기획*에서 *실제 빌드*로 넘어온 날. 작업 환경: 아이폰 + Termius(SSH) + DigitalOcean Ubuntu 24.04 VPS 이 문서는 *복습용 과정 기록*이다 — 뭘, 어떻게, 어떤 순서로, 왜 했는지.

한눈에 — 오늘 한 4가지

- 8-K 트레이딩 봇 은퇴 (옛 환경 정리)
- Java 환경 구축 (JDK 21 + 첫 프로그램)
- Git + GitHub 연동 (SSH 인증 + 첫 커밋 + 공개 잔디)
- gh CLI 설치 (향후 repo 생성 자동화)

결과: 돌아가는, 공개된, 버전관리되는 개발 환경을 맨손으로 세웠다.

Phase 0 — 8-K 트레이딩 봇 은퇴

왜: 새 백엔드 학습 환경을 위해 옛 봇을 깨끗이 정리. (봇이 계속 돌면 Claude API 비용도 샘.)

한 일:

- systemd 서비스 중지 + 자동시작 해제 `bash systemctl stop 8k-trader systemctl disable 8k-trader # 재부팅해도 안 켜짐 systemctl status 8k-trader # → inactive (dead) 확인`
- 봇 파일 *보관*(삭제 아님 — 되돌릴 수 있게) `bash mkdir -p ~/_archive mv /opt/8k-trader* /opt/8k-backtest ~/_archive/ # /opt의 봇 폴더들 mv ~/main.py ~/sec_fetcher.py ~/install.sh ~/API ~/_archive/ # 홈의 봇 파일들 ls -la ~ # 홈 깨끗한지 + 숨은 파일 확인`
- (권고) Alpaca / Anthropic 콘솔에서 옛 API 키 revoke — 비용·보안

배운 것: - `systemctl` (서비스 중지/비활성), 서비스는 stop만으론 재부팅 시 다시 켜짐 → `disable` 필요 - 삭제 대신 `mv` 로 보관 = 안전한 정리 - 페이지저(less) 화면에서 빠져나오기 = `q`

Phase 1 — Java 환경 구축

왜: JDK 21 LTS(Temurin) 채택 — 한국 핀테크 프로덕션 주류(17~21), 채용 요건과 정렬, 튜토리얼·라이브러리 호환 최대. 도구 선택: `apt` 대신 **SDKMAN** — JDK 여러 버전 전환 + Gradle·Kotlin까지 한 도구로 관리. 로드맵(Java→Kotlin, Gradle)에 정확히 맞춤.

한 일:

- SDKMAN 의존성 + 설치 `bash sudo apt install -y zip unzip curl curl -s "https://get.sdkman.io" | bash source "$HOME/.sdkman/bin/sdkman-init.sh" sdk version`
- JDK 21 (Temurin) 설치 `bash sdk list java # 페이지에서 /tem 검색 → 21.0.11-tem 찾을 sdk install java 21.0.11-tem java -version # 21.0.11 확인 javac -version # 컴파일러도 확인`
- 첫 Java 프로그램 `bash mkdir -p ~/java-practice && cd ~/java-practice nano Hello.java java public class Hello { public static void main(String[] args) { System.out.println("Hello from VPS!"); } } bash javac Hello.java && java Hello # → "Hello from VPS!"`

배운 것: - SDKMAN으로 JDK 버전 관리 (`sdk list` / `sdk install`) - **클래스명 = 파일명** 규칙 (`Hello.java` ↔ `public class Hello`) - 편집 → 컴파일 → 실행 루프 (`nano` → `javac` → `java`) - 컴파일(`javac` , `.class` 생성)과 실행(`java`)은 별개 단계 - 폰 작업 팁: Termius **extra keys**(Ctrl), nano 저장 `Ctrl+O` /종료 `Ctrl+X` , ↑로 직전 명령 재호출

Phase 2 — Git + GitHub 연동

왜: 버전관리 + *공개 잔디*(평가관이 보고, "일찍 시작" 효과). 코드를 GitHub에 쌓는다. 인증 방식 결정: **SSH 키(ed25519)** — 만료 관리 불필요(매일 커밋에 유리), AWS EC2 등 서버 인증에 그대로 전이되는 기초 스킬.

한 일:

- git 설정 `bash git config --global user.name "Taewoo-Won" git config --global user.email "288912277+Taewoo-Won@users.noreply.github.com" git config --global init.defaultBranch main git config --global --list # 확인` - 이메일은 **GitHub noreply** 주소 사용 → (a) 개인 이메일 비공개 (b) 잔디 확실해 찍힘 - 주의했던 점: 처음에 예시 플레이스홀더 "**표시할이름**" 을 그대로 넣었다가 실제 값으로 교정
- SSH 키 생성 (+ 보안 점검 차 1회 재생성) `bash ssh-keygen -t ed25519 -C "github-vps" # passphrase 빈 값` - 중간에 "내 키가 노출된 거 아닌가" 점검 → *공개키는 원래 공개용*임을 확인, 안심 차원에서 키 재생성: `bash rm ~/.ssh/id_ed25519 ~/.ssh/id_ed25519.pub ssh-keygen -t ed25519 -C "github-vps"`
- 공개키 GitHub 등록 `bash cat ~/.ssh/id_ed25519.pub # ssh-ed25519 ... 한 줄` → GitHub 웹 → Settings → SSH and GPG keys → New SSH key → **Authentication Key**로 추가
- 연결 테스트 `bash ssh-keyscan -t ed25519 github.com >> ~/.ssh/known_hosts # 호스트 키 등록 ssh -T git@github.com # → "Hi Taewoo-Won! ..." - GitHub 호스트 키 지문( SHA256:DiY3wvvV6... )이 GitHub 공식 값과 일치하는지 검증 후 신뢰`
- 로컬 repo + 첫 커밋 `bash echo "*.class" > .gitignore # 빌드 산물(.class)은 커밋 제외 git init git add . git status # .gitignore, Hello.java staged / Hello.class 제외 확인 git commit -m "first commit: hello world on VPS" # → c6db593`

6. GitHub repo 생성 + push - GitHub 웹 → New repository → 이름 `java-practice` → README·.gitignore·license **전부 해제**(빈 repo, 안 그러면 push 충돌) `bash`
`git remote add origin git@github.com:Taewoo-Won/java-practice.git` `git push -u origin main` # → [new branch] main -> main - **겪은 일**: repo를 먼저 안 만들고 push해서 `Repository not found` → repo 생성 후 재push 성공
7. 검증 (완료 기준) - repo에 `Hello.java` 보임 ✓ - 프로필 잔디 초록 1칸 ✓ (repo가 **Public**이라 표시됨 — Private이면 공개 프로필에 안 뜸)

배운 것: - **공개키 vs 개인키** = 자물쇠(공개) vs 열쇠(비밀). 노출돼서 문제 되는 건 *개인키*뿐 - **HTTPS vs SSH** = GitHub 들어가는 두 개의 문. HTTPS는 비번/토큰, SSH는 키 - **remote / origin** = 로컬 git이 저장해둔 "GitHub 주소 별명" (연락처 저장처럼) - **호스트 키 지문 검증** = `yes` 치기 전 중간자 공격 방어 - `.gitignore` = 빌드 산물 등 커밋 제외 / `git init` · `add` · `commit` · `push` 의 역할

Phase 3 — gh CLI 설치 (보너스)

왜: 다음 repo부터 Safari 웹 폼 없이 *터미널 한 줄*로 생성. 폰 워크플로 마찰 최소화.

한 일:

```
sudo apt install gh -y # gh 2.45.0
gh auth login
# → GitHub.com / SSH / Skip(키 이미 등록) / Login with web browser
# → device flow: 코드(9842-410C) → Safari github.com/login/device → Authorize
gh auth status # → Logged in as Taewoo-Won
```

배운 것: - **gh ≠ git**. git = 버전관리(add/commit/push, *매일*). gh = GitHub 관리(repo 생성·PR·이슈, 원래 웹에서 하던 것) - **device flow** = 터미널에 비번 안 치고 일회용 코드로 브라우저서 인증 - 향후 새 repo = `gh repo create 이름 --public --source=. --remote=origin --push` *한 줄* (생성+연결+push)

오늘의 산출물 (완료 상태)

- ✓ **Java** (JDK 21 Temurin) 환경 + 편집/컴파일/실행 루프
- ✓ **git** 버전관리 + SSH 인증
- ✓ **GitHub** 공개 repo + 첫 커밋 + 잔디
- ✓ **gh CLI** 설치/인증
- 🔗 공개 repo: `github.com/Taewoo-Won/java-practice`
- 🔗 첫 커밋: `c6db593` — "first commit: hello world on VPS"

앞으로 일상 루프 (매일 이거)

코드 수정 후:

```
git add .
git commit -m "의미 있는 메시지" # "fix" 남발 X, "implement WAL append" 처럼
git push # → 잔디 자동 누적
```

폰에선 ↑ 로 직전 명령 재호출. `git status` (현재 상태), `git log --oneline` (이력).

다음 작업

Java 기본 문법 학습 (클래스·컬렉션·예외·제네릭) — Spring/JPA로 가기 전 기초. → 새 채팅 주제: "Java 기본 문법 학습 루트 + 첫 실습 과제 1개"

오늘 배운 핵심 개념 (한 줄 사전)

개념	한 줄 정의
<code>systemctl</code>	리눅스 서비스 시작/중지/비활성 관리. stop ≠ disable
페이지(<code>less</code>)	긴 출력 화면. <code>q</code> 로 나옴
SDKMAN	JDK·Gradle·Kotlin 버전 관리 도구
클래스명=파일명	Java 규칙. <code>Hello.java</code> ↔ <code>public class Hello</code>
공개키/개인키	자물쇠(공개 OK) / 열쇠(절대 비밀)
HTTPS vs SSH	GitHub 연결 두 방법. 비번 vs 키
remote/origin	저장해둔 GitHub 주소 별명
호스트 키 지문	접속 대상이 진짜인지 검증 (MITM 방어)
<code>.gitignore</code>	커밋에서 제외할 파일 목록
gh vs git	GitHub 관리 도구 vs 버전관리
device flow	일회용 코드로 브라우저 인증

한 줄 회고

며칠 전까지 전부 *기*획이었다. 오늘 처음으로 *손*으로 *만*들었다. 환경·버전관리·공개 잔디까지 — 9개월 여정에서 가장 중요한 한 걸음은 **시작**이었고, 그걸 오늘 했다.